Fault-Tolerant Self-Checking Framework for Vision-Language Models

XIN FAN LI
Fardin Khandaker

May 16, 2026

Abstract

Vision-Language Models (VLMs) are being increasingly deployed in real-world autonomous learning scenarios and multi-component AI systems; however, these systems remain susceptible to erroneous outputs. This paper investigates a lightweight, lightweight self-checking framework designed to detect inconsistencies in image description tasks. The framework operates by first utilizing the BLIP model to generate image descriptions, and subsequently injecting synthetically generated errors through controlled sentence-construction and substitution operations. A discriminative module then employs Sentence-BERT's cosine similarity metric to assess the semantic consistency of the descriptions. Experimental results based on 100 image-description pairs yielded an average similarity score of 0.856, with a standard deviation of 0.080. When a similarity threshold of 0.7 was applied, the framework succeeded in detecting only 2 percent of the injected errors. This outcome suggests that the embeddings generated by Sentence-BERT are relatively insensitive to subtle lexical perturbations—particularly when the overall syntactic structure of the sentences remains similar. These findings not only highlight the feasibility of similarity-based error detection methods within lightweight vision-language verification systems but also underscore the inherent limitations of such approaches.

1 Introduction

1.1 Motivation

In modern vision-language systems, the importance of self-monitoring and fault-tolerance mechanisms is becoming increasingly evident, particularly in tasks such as image captioning, visual question answering, and robotic perception. Although vision-language models are capable of generating highly fluent outputs, they remain susceptible to semantic inconsistencies, hallucinations, or erroneous object descriptions.

Detecting such inconsistencies is often extremely challenging, as many sentence embedding models prioritize global semantic similarity over the identification of precise object-level consistency. Consequently, this paper explores whether semantic similarity can serve as a lightweight self-monitoring mechanism for identifying semantic perturbations within generated descriptions.

1.2 Problem Statement

This paper investigates whether semantic similarity measures can detect artificially injected faults in image captions generated by vision-language models.

- Whether semantic similarity can identify inconsistent captions

- How sensitive Sentence-BERT embeddings are to local word perturbations
- Whether lightweight similarity-based critics can serve as a basic fault detection mechanism

1.3 Contributions

Our main contributions are:

1. A lightweight self-checking framework for analyzing caption consistency in vision-language systems
2. A synthetic fault injection mechanism using controlled word replacement
3. An experimental evaluation of Sentence-BERT similarity sensitivity under semantic perturbations
4. A discussion of the limitations of similarity-based fault detection for subtle object-level errors

1.4 Paper Organization

The remainder of this paper is organized as follows. Section 2 reviews related work in multi-agent systems, vision-language models, and fault tolerance. Section 3 presents the proposed self-checking framework and semantic similarity mechanism. Section 4 describes the experimental setup and evaluation methodology. Section 5 presents the experimental results and visualization analysis. Section 6 discusses the limitations and implications of the framework. Section 7 concludes the paper and outlines future research directions.

2 Related Work

2.1 Multi-Agent Systems

Multi-agent systems (MAS) have been the subject of extensive research worldwide within the fields of artificial intelligence and robotics. Shoham and Leyton-Brown [6] established a comprehensive theoretical foundation for MAS, encompassing topics such as coordination, collaboration, and communication protocols.

In the study of fault-tolerant MAS, redundancy and consistency checking are commonly employed strategies for identifying potential agent failures. Multiple components or verification modules collectively evaluate the same task, and by comparing their respective outputs, analyze patterns of consistency or inconsistency in the results.

Our work draws inspiration from the aforementioned concepts of fault-tolerant verification, applying them to the semantic consistency checking of vision-and-language image description systems.

2.2 Vision-Language Models

Recent years have seen rapid advances in vision-language models. BLIP (Bootstrapping Language-Image Pre-training) [1] introduced a unified framework for vision-language understanding and generation, achieving strong performance on image captioning and VQA. CLIP [5] learned visual concepts from natural language supervision, enabling zero-shot transfer to many tasks. More recently, LLaVA [4] demonstrated instruction-following capabilities for vision-language tasks.

Despite their success, VLMs are known to be brittle. They can hallucinate objects not present in images, produce inconsistent outputs under minor input perturbations, and fail on simple reasoning tasks. These vulnerabilities become critical in multi-agent settings where failures can propagate.

2.3 Fault Tolerance in AI Systems

Fault tolerance has long been studied in distributed systems and robotics. Traditional approaches include redundancy, checkpointing, and voting mechanisms. In the context of neural networks, ensemble methods combine multiple models to improve accuracy and robustness.

However, few works explicitly address fault tolerance in multi-agent VLM systems. Most existing research focuses on improving single-model performance rather than detecting failures in multi-agent interactions. Our work builds on redundancy and cross-checking strategies but adapts them to the unique challenges of natural language outputs from VLMs.

2.4 Gap in Literature

The key gap we identify is the lack of lightweight fault detection mechanisms for multi-agent VLM systems. While ensemble methods combine outputs to improve accuracy, they do not explicitly identify which outputs are potentially faulty. Our approach focuses specifically on detecting disagreements as indicators of faults, which can then trigger recovery mechanisms.

3 Methodology

3.1 Overview

Our proposed framework consists of three functional components: a generator, a perturbation module, and a critic.

3.2 Framework Components

3.2.1 Generator

The generator is responsible for producing an initial caption for the input image. It uses deterministic generation to provide a stable baseline. Formally, given an input image I, the generator produces:

$$c_g = G(I)$$

where G is the generator model and c_g is the generated caption.

3.2.2 Perturbation Module

Instead of using a second independent vision-language model, our framework simulates caption faults through synthetic perturbation. After the generator produces an image caption, random words are replaced with semantically inconsistent tokens such as 'dog", 'car", 'tree", or 'building".

Formally, given a generated caption c_g, the verifier produces a modified caption:

$$c_v = P(c_g)$$

where P represents the perturbation function.

This approach allows controlled evaluation of whether the critic module can detect semantic inconsistencies using similarity scores. Although this setup does not represent a fully independent multi-agent architecture, it provides a computationally efficient method for studying fault sensitivity in semantic embedding space.

3.2.3 Critic

The critic compares the two captions and decides whether they are semantically similar enough. If not, a potential fault is flagged. The critic computes:

$$s = \mathrm{Sim}(c_g, c_v)$$

where Sim is a semantic similarity function. The fault detection decision is:

$$\text{fault} = \begin{cases} \text{True} & \text{if } s < \tau \\ \text{False} & \text{otherwise} \end{cases}$$

where τ is a predefined threshold (we use $\tau = 0.7$ in our experiments).

3.3 Semantic Similarity Computation

We use Sentence-BERT [2] to compute semantic similarity between captions. Sentence-BERT encodes each text into a fixed-dimensional vector such that semantically similar texts have similar vectors. Specifically:

$$\mathrm{Sim}(c_g, c_v) = \frac{\mathbf{e}_g \cdot \mathbf{e}_v}{\|\mathbf{e}_g\|\|\mathbf{e}_v\|}$$

where $\mathbf{e}_g = \mathrm{SBERT}(c_g)$ and $\mathbf{e}_v = \mathrm{SBERT}(c_v)$ are the embeddings of the two captions. This is the cosine similarity, ranging from -1 to 1, though in practice for natural language it ranges from 0 to 1.

Algorithm 1 Fault Detection Algorithm

Require: Input image I, threshold $\tau = 0.7$
Ensure: Fault flag f, similarity score s

```
c_g ← Generator(I)
c_v ← InjectFault(c_g)
e_g ← SBERT(c_g)
e_v ← SBERT(c_v)
s ← cosine_similarity(e_g, e_v)
if s < 0.7 then
    f ← True
else
    f ← False
end if
return (f, s)
```

4 Experimental Setup

4.1 Dataset

We conduct experiments using a small collection of publicly accessible images obtained from Unsplash. The dataset contains five representative images, including scenes with animals, indoor environments, furniture, and human activities.To increase the number of evaluation samples while keeping computational cost manageable, the five images were repeated to generate 100 image-caption pairs.This lightweight setup allows rapid experimentation and controlled evaluation of synthetic fault injection without requiring large-scale dataset preprocessing.

4.2 Dataset Feasibility Analysis

Before conducting experiments, we evaluated whether a lightweight image collection would be sufficient for testing semantic similarity-based fault detection.

Publicly accessible Unsplash images were selected because they contain diverse visual scenes, including animals, indoor environments, furniture, and human activities. These scenes are suitable for evaluating whether semantic perturbations in generated captions can be detected using Sentence-BERT similarity.

Due to computational limitations in the Google Colab environment, the experiment was intentionally designed as a lightweight feasibility study using five representative images repeated across 100 samples.

Although the dataset is relatively small and not statistically comprehensive, it provides a controlled environment for analyzing the sensitivity of semantic similarity under synthetic caption perturbation.

Future work will extend the evaluation to larger benchmark datasets and more diverse image collections.

4.3 Models

Due to computational constraints, we use relatively lightweight models:

4.3.1 BLIP-base

BLIP-base [1] is a 140M parameter model for image captioning. It uses a vision transformer (ViT) as the image encoder and a BERT-based text decoder. We use the version fine-tuned on COCO captioning, which achieves a CIDEr score of approximately 113 on the COCO test set.

4.3.2 Sentence-BERT (all-MiniLM-L6-v2)

This is a 22M parameter model for generating sentence embeddings. It maps text to 384-dimensional vectors and achieves strong performance on semantic textual similarity benchmarks. We use it because it is fast (CPU-compatible) and reliable.

4.4 Evaluation Metrics

We report the following metrics:

- Mean similarity: Average cosine similarity across all samples
- Standard deviation: Spread of similarity scores
- Minimum similarity: Lowest similarity score observed
- Maximum similarity: Highest similarity score observed
- Fault detection rate: Percentage of samples with similarity below the threshold of 0.7

4.5 Hardware

Experiments were conducted on Google Colab with the following specifications:

- GPU: NVIDIA T4 (16GB VRAM)
- CPU: Intel Xeon (2.2 GHz, 2 cores)
- RAM: 13GB
- Runtime: Approximately 10 minutes for 100 image-caption pairs

5 Results

5.1 Quantitative Results

Table 1 presents the overall results of our experiment on 100 image-caption pairs.

Table 1: Overall fault detection results

Metric	**Value**
Total samples	100
Detected faults	2
Undetected faults	98
Fault detection rate	2%

Table 2: Similarity statistics

Metric	**Value**
Mean similarity	0.856
Standard deviation	0.080
Minimum similarity	0.643
Maximum similarity	1.000

The results show that most modified captions remained highly similar to the original captions despite injected perturbations. Only two samples fell below the threshold of 0.7, resulting in a very low fault detection rate.

5.2 Visualizations

Figure 1 shows the distribution of semantic similarity scores produced by the framework after synthetic fault injection.

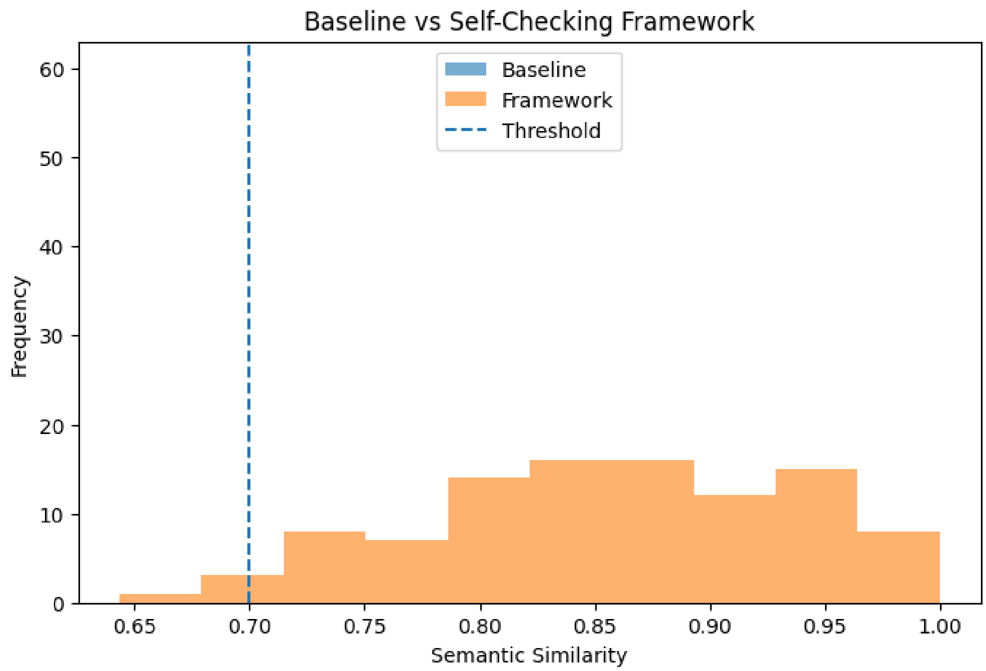

Figure 1: Distribution of semantic similarity scores with threshold at 0.7.

Most similarity scores are concentrated between 0.75 and 1.0, indicating that the injected captions remain semantically close to the original captions despite word replacement.

Only a small number of samples fall below the threshold of 0.7. This demonstrates that Sentence-BERT embeddings are relatively insensitive to local lexical perturbations when overall sentence structure remains similar.

The visualization also suggests that simple word substitution is insufficient for generating strongly contradictory captions in semantic embedding space.

6 Discussion

6.1 Interpretation of Results

The experimental results reveal that Sentence-BERT similarity is relatively insensitive to small local perturbations. Replacing individual words such as 'cat" with 'dog" or 'table" with 'car" often still produced similarity scores above 0.8.

This behavior suggests that sentence embedding models prioritize overall sentence structure and global semantic context rather than exact object-level consistency. As a result, similarity-based critics may fail to detect subtle hallucinations or object substitution errors in vision-language systems.

Although the framework successfully detected a small number of strongly inconsistent captions, most injected faults remained undetected under the current threshold setting.

6.2 Effect of Threshold

The choice of threshold τ significantly affects fault detection sensitivity. Under the current threshold of 0.7, only two samples were classified as faults because most similarity scores remained relatively high after word substitution.

Lowering the threshold would further reduce the number of detected faults, while increasing the threshold would make the framework more sensitive to semantic perturbations. Selecting an appropriate threshold therefore depends on the balance between false positives and false negatives in a specific application setting.

6.3 Challenges

Two main challenges emerged from our experiment.

First, vision-language models remain computationally expensive, limiting the scale and diversity of our experiments. Although the current study uses 100 image-caption pairs, the underlying image set is relatively small and repeated across samples. Running the same experiment on 500+ images would require significantly more resources.

Second, defining what counts as a "fault" in generated outputs is inherently subjective. Two captions may describe the same scene while emphasizing different objects or attributes. Whether such differences should be considered faults depends on the application context. A high-precision system may require exact object consistency, while a more flexible system may tolerate minor semantic variation.

6.4 Limitations

Our current implementation has several limitations:

- The verifier does not use an independent vision-language model and instead relies on synthetic fault injection
- The evaluation uses a small repeated image set rather than a large benchmark dataset
- Sentence-BERT similarity alone is insufficient for detecting subtle object-level inconsistencies
- The similarity threshold was manually selected rather than optimized using validation data
- The framework was evaluated only on image captioning tasks

7 Conclusion and Future Work

7.1 Feasibility

The complete implementation, including model loading, caption generation, similarity computation, and experiment scripts, is available in a GitHub repository.

The repository is organized into modules for data processing, model inference, and evaluation, allowing easy replication of results.

GitHub repository: https://github.com/lxf6323210-hash/59866-final-project

7.2 Conclusion

This paper investigated a lightweight self-checking framework for semantic fault detection in image captioning systems. Using BLIP-generated captions, synthetic perturbation, and Sentence-BERT similarity analysis, we evaluated whether semantic similarity can identify injected caption inconsistencies.

Experimental results showed that most injected faults were not detected because Sentence-BERT embeddings remained highly similar despite local word substitutions. This finding demonstrates that semantic similarity alone is insufficient for reliable object-level fault detection in vision-language systems.

Future work should incorporate stronger verifier models, object-level grounding methods, and multimodal consistency checking to improve fault sensitivity and robustness.

7.3 Future Work

Several directions remain for future investigation:

- **Different architectures**: Use different base models for generator and verifier (e.g., BLIP for generator, LLaVA for verifier)
- **Task diversity**: Test on visual question answering (VQA) and image-text retrieval
- **Scale**: Evaluate on 500+ images to obtain statistically significant results
- **Threshold tuning**: Optimize τ using a validation set with ground truth labels
- **Recovery mechanisms**: Implement fallback strategies when faults are detected (e.g., query a third agent, request human input)

References

[1] J. Li, D. Li, C. Xiong, and S. Hoi, "BLIP: Bootstrapping Language-Image Pre-training for Unified Vision-Language Understanding and Generation," in *ICML*, 2022.

[2] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *EMNLP*, 2019.

[3] T. Lin, M. Maire, S. Belongie, L. Bourdev, R. Girshick, J. Hays, P. Perona, D. Ramanan, C. L. Zitnick, and P. Dollár, "Microsoft COCO: Common Objects in Context," in *ECCV*, 2014.

[4] H. Liu, C. Li, Q. Wu, and Y. J. Lee, "Visual Instruction Tuning," in *NeurIPS*, 2023.

[5] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever, "Learning Transferable Visual Models From Natural Language Supervision," in *ICML*, 2021.

[6] Y. Shoham and K. Leyton-Brown, *Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations*. Cambridge University Press, 2009.

[7] F. P. A. Coolen, *Fault Tolerance*. Springer, 2012.

[8] A. Radford et al., "Learning Transferable Visual Models From Natural Language Supervision," ICML, 2021.

[9] S. Antol, A. Agrawal, J. Lu, M. Mitchell, D. Batra, C. L. Zitnick, and D. Parikh, "VQA: Visual Question Answering," in *ICCV*, 2015.

[10] T. G. Dietterich, "Ensemble Methods in Machine Learning," in *Multiple Classifier Systems*, 2000.

8 Project Timeline and Task Allocation

8.1 Timeline

Week 1: Literature review on multi-agent systems and vision-language models; formulation of the research question.

Week 2: Design of the self-checking framework and semantic consistency mechanism (generator, verifier, critic) and selection of models (BLIP and Sentence-BERT).

Week 3: Implementation of the system, including image caption generation and semantic similarity computation.

Week 4: Execution of experiments on sample images and initial analysis of results.

Week 5: Visualization of results and evaluation of system performance; dataset feasibility validation.

Week 6: Refinement of methodology, report writing, and final revision.